

Make a 4x4 Keypad Talk

Wire a few keys, win a light show, then finish the number pad

Every correct key earns an immediate D13 celebration.

Lesson	016 — component	Time	45–75 minutes
Prerequisites	Lessons 001–015	Board	Arduino Mega 2560
Payoff	D13 patterns for every active key	Part	authorized 4x4 keypad
Safety	E1: USB-powered passive input circuit only		

1 Mission

Build one operator-input path:

4x4 keypad in 12-key mode → `MatrixKeypad` → `KeypadSnapshot` → D13 evidence.

`MatrixKeypad` owns and scans seven active pins. The real specimen is an eight-wire 4x4 keypad: this first adventure uses all four rows and the three number-pad columns while the identified A/B/C/D column is safely parked. Its composed `Keypad` interpreter debounces the 12-bit sample, maps one position to a semantic key, and rejects chords until every key is released. Time enters only through `update(now)`.

Boundary. This lesson teaches input interpretation. It is not a lock, credential system, or anti-ghosting keypad. A diode-less matrix can produce ambiguous multi-key observations, so every chord is rejected.

1.1 Learning outcomes

- explain row selection and column pull-ups;
- separate electrical scanning from deterministic interpretation;
- predict debounce, press, release, and invalid-chord events;
- distinguish acquisition evidence from safe-state evidence;
- use RAII cleanup without exceptions, heap allocation, or blocking delay.

2 Parts and safety

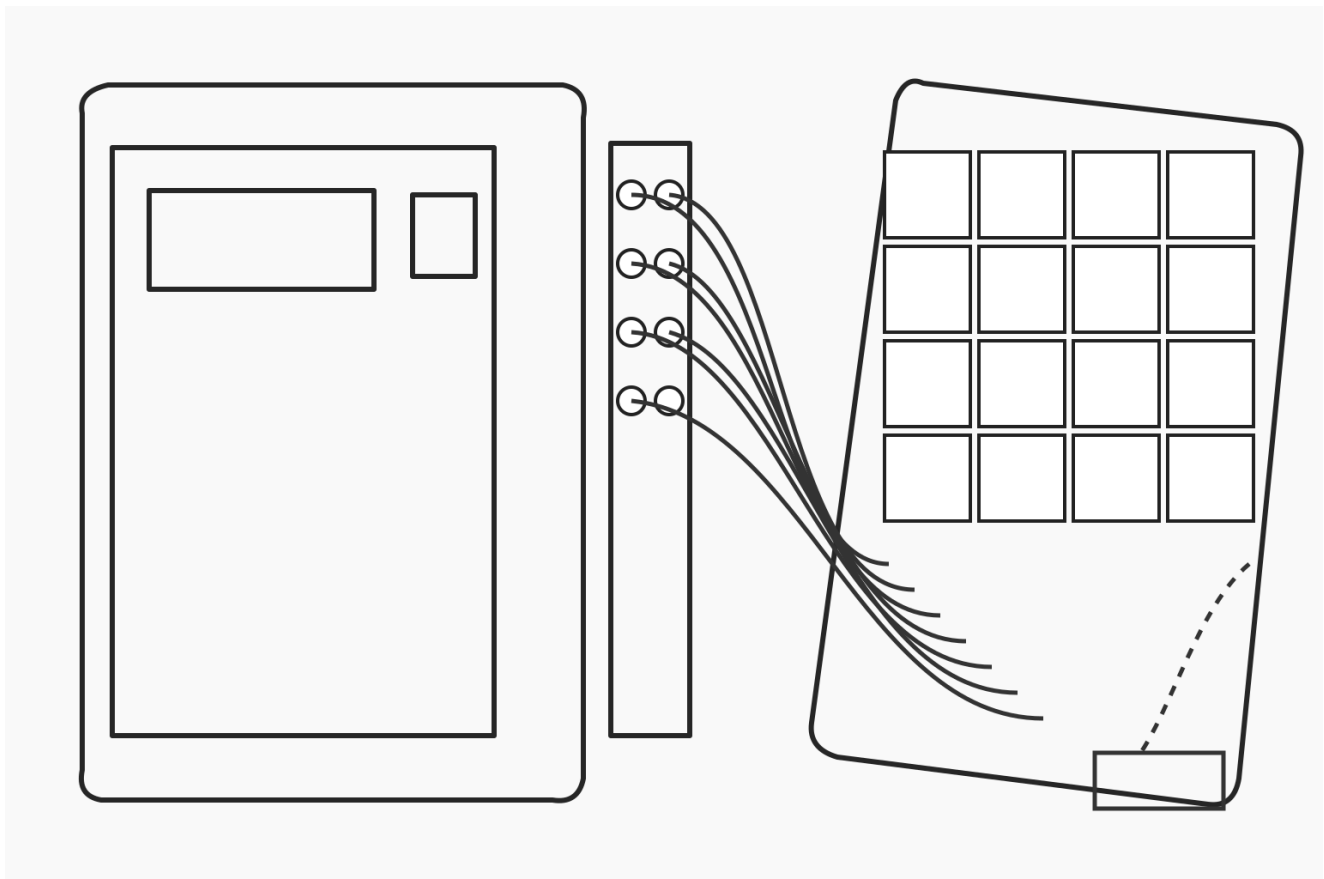
Part	Requirement
Arduino Mega 2560	USB-powered board
4x4 membrane keypad	Passive switch matrix with identified eight-wire tail
Seven jumpers	D22 through D28; direct wiring, no breadboard needed
Insulating tape	Covers the unused A/B/C/D-column conductor by itself
Logic probe or oscilloscope	Optional TP-R0 observation; high-impedance input

Remove USB power before moving wires or attaching a probe. Identify the exact eight-wire tail order with an unpowered continuity test; printed connector order is not standardized. Label R0–R3 and C0–C3. Cover C3 by itself. No keypad conductor connects directly to 5 V or GND.

3 Authoritative wiring

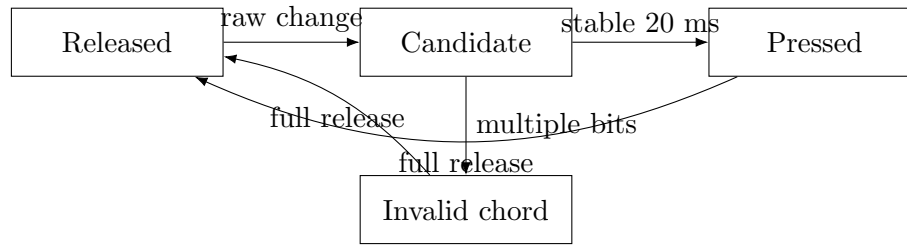
Signal	Mega pin	Keypad contact
Row0 / TP-R0	D22	row containing 1, 2, 3
Row1	D23	row containing 4, 5, 6
Row2	D24	row containing 7, 8, 9
Row3	D25	row containing *, 0, #
Column0	D26	column containing 1, 4, 7, *
Column1	D27	column containing 2, 5, 8, 0
Column2	D28	column containing 3, 6, 9, #
Column3	parked	column containing A, B, C, D; insulated, no connection
Operator evidence	D13	board LED; no external wire

This is direct wiring: the keypad does not plug into the breadboard. On the Mega paired side header, D22/D23 form the first pair, then D24/D25, D26/D27, and D28/D29. Follow that physical zig-zag and leave D29 empty. During each scan, exactly one row becomes an output driven low. Active columns remain inputs with internal pull-ups. A closed key pulls its column low. The selected row immediately returns to input before the next row is selected.



Alternative text: pencil orientation plate with a Mega on the left and a 4x4 keypad on the right. The Mega side header shows its actual paired D22/D23 through D28/D29 geometry. Seven labeled wires connect D22–D28 to R0–R3 and C0–C2; C3, the A/B/C/D column, ends in its own taped parking tab. There is no breadboard because every connection is direct.

4 Scan and interpretation



A stable single bit produces one `pressEvent`. Holding the key produces no repeats. A full stable release rearms the next press. A chord that collapses to one key remains disarmed until complete release.

Observation	D13 pattern	Interpretation
Initialization	one short pulse	keypad and LED claims succeeded
Digit 1–9	one to nine short pulses	accepted semantic digit
Digit 0	ten short pulses	accepted zero, distinct from ready
*	one long pulse	accepted star
#	two long pulses	accepted hash
Invalid chord	repeating two-short group	release every key before retry
Scan/update fault	repeating three-short group	stop relying on input

5 Narrative code

```

void loop ()
{
  if (!running)
  {
    return;
  }

  const adk::TimePoint      now      (millis ());
  const adk::KeypadSnapshot observation = observeKeypad      (now);
  const OperatorDecision    decision  = decideOperatorEvent (observation);

  if (!showOperatorEvidence (now, decision))
  {
    stopSafely ();
  }
}

```

The canonical sketch acquires the keypad, then the evidence LED. On a partial failure it releases the earlier object. The loop always reads “observe, decide, actuate.” D13 patterns are nonblocking, so scanning continues while a result is displayed. `stopSafely()` shuts down in reverse order.

6 Build, play, expand

6.1 Win 1: wake-up wink

Upload before wiring the keypad. One quick D13 wink means the sketch is ready. Unplug USB before adding wires.

6.2 Win 2: make 1, 2, 3 answer

Identify the tail with continuity mode, label all eight conductors, and park C3. Connect R0–R3 and C0–C2 exactly as drawn. Restore USB. Keys 1, 2, and 3 answer with one, two, and three short flashes. A wrong order means a label or wiring mismatch; fix it with power removed.

6.3 Win 3: play the whole number pad

Try every digit. Zero earns ten quick flashes; star earns one long flash and hash earns two. Hold one key down: it celebrates once. Release it fully and it is ready to celebrate again. Pressing a muddled chord gives paired flashes; release everything and keep playing.

6.4 Remix

Make a three-key rhythm, spell a favorite number, or challenge someone to identify the key by watching D13 alone. Reset restarts immediately. Unplug USB when finished; the passive keypad stores no energy.

7 Diagnosis

Observation	Check
No ready pulse and no TP-R0 pulse	pin conflict, initialization status, and D13 ownership
Ready pulse but no accepted key	tail order, continuity, and row/column mapping
Wrong digit	logical row/column order; do not rewrite the semantic map to hide wiring
Paired pattern for one key	unintended bridge or ambiguous tail identification
Three-short pattern	update status and lifecycle; do not treat the snapshot as input
TP-R0 continues after shutdown	safe-state failure; remove USB power and record it

8 What the library quietly guarantees

The reusable code rejects duplicate pins, rolls back partial acquisition, debounces without delay, rejects ambiguous chords, survives clock wraparound, and returns the active pins to high impedance on shutdown. Those checks run behind the scenes so the lesson can stay focused on wiring, pressing keys, and making light.